# PX4 + Gazebo Harmonic Integration Guide for Custom X650 UAV (v5)

End-to-end procedure to simulate a custom quadcopter (X650) in Gazebo Harmonic using the PX4 Control Stack on Ubuntu 24.04 LTS. Includes Xacro→URDF→SDF, portable model setup, PX4 airframe, launch, tests, and fixes.

## 1. Prerequisites

Ubuntu 24.04 LTS; Gazebo Harmonic (gz-sim 8.x); PX4-Autopilot (cloned with submodules). Optional: ROS 2 Jazzy for bridging topics.

```
sudo apt update
sudo apt install git build-essential python3-vcstool
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd PX4-Autopilot
make px4_sitl gz_x500   # baseline verification
```

## 2. Xacro → URDF

Expand your Xacro into URDF and validate it has a root:

```
# Expand Xacro to URDF
ros2 run xacro xacro uav.urdf.xacro > uav.urdf

# Validate URDF (must have <robot> root)
check_urdf uav.urdf
```

## 3. URDF → SDF (for Gazebo Harmonic)

Convert and validate the SDF. Remove legacy tags to silence warnings:

```
# Convert and pretty-print
gz sdf -p uav.urdf > model.sdf

# Validate against SDF schema/semantics
gz sdf -k model.sdf

# Remove legacy tags not in SDF (harmless but noisy)
sed -i '/<gz_frame_id>.*<\/gz_frame_id>/d' model.sdf
```

Do not use Classic-only plugins in Gazebo Harmonic (e.g., libMavlinkInterface.so). PX4's GZ bridge is built into px4.

## 4. Build a Portable Gazebo Model Folder

Create the model folder and ensure all mesh filenames are lowercase to avoid case issues:

```
# Folder layout
PX4-Autopilot/Tools/simulation/gz/models/x650/
■■■ model.config
■■■ model.sdf
■■■ meshes/
    ■■■ base_link.stl
```

```
■■■ back_prop_ccw_link.stl
■■■ back_prop_cw_link.stl
■■■ front_prop_ccw_link.stl
■■■ front_prop_cw_link.stl
■■■ depth_camera_link.stl
■■■ gps_module_link.stl
■■■ lidar_link.stl
```

Use model-relative URIs and fix absolute file paths:

```
# Ensure SDF uses model:// URIs (portable)
<uri>model://x650/meshes/base_link.stl</uri>

# Rename uppercase .STL to lowercase .stl (Linux is case-sensitive)
cd PX4-Autopilot/Tools/simulation/gz/models/x650/meshes
for f in *.STL; do mv "$f" "$(basename "$f" .STL).stl"; done

# Fix any absolute file:/// URIs in model.sdf
sed -i 's|<uri>file:///[^<]*/meshes/|<uri>model://x650/meshes/|g' ../model.sdf
```

## 5. SDF Essentials for Harmonic (Sensors & Motors)

Keep IMU, magnetometer, barometer, GPS, camera/LiDAR as needed. Do NOT add a duplicate Sensors system plugin at model level; the world loads it automatically.

Add the MulticopterMotorModel plugin block for each rotor (repeat with unique joint/link/motorNumber):

```
<!-- Repeat this block for ALL motors, changing
jointName/linkName/turningDirection/motorNumber -->
<plugin name='gz::sim::systems::MulticopterMotorModel'
filename='gz-sim-multicopter-motor-model-system'>
  <jointName>front_prop_ccw_joint</jointName>
  <linkName>front_prop_ccw_link</linkName>
  <turningDirection>ccw</turningDirection>
  <timeConstantUp>0.0125</timeConstantUp>
  <timeConstantDown>0.025</timeConstantDown>
  <maxRotVelocity>1200</maxRotVelocity>
  <motorConstant>3.0e-05</motorConstant>
  <momentConstant>0.016</momentConstant>
  <commandSubTopic>command/motor_speed</commandSubTopic>
  <motorNumber>0</motorNumber>
  <rotorDragCoefficient>8.06428e-05</rotorDragCoefficient>
  <rollingMomentCoefficient>1e-06</rollingMomentCoefficient>
  <rotorVelocitySlowdownSim>30</rotorVelocitySlowdownSim>
  <motorType>velocity</motorType>
</plugin>
```

Remove any Classic-only bridge plugin from model.sdf:

```
<!-- Remove this (Classic only) -->
<!-- <plugin name="mavlink_interface" filename="libMavlinkInterface.so"> ... </plugin> -->
```

## 6. PX4 SITL Airframe (GZ Style)

Create a custom airframe under ROMFS and package it in the build. Start from Gazebo x500 template (4001_gz_x500).

```
cd PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes
cp 4001_gz_x500 4229_gz_x650

# The file must start with:
#!/bin/sh
```

Then set variables and geometry (example):

```
. ${R}etc/init.d/rc.mc_defaults

PX4_SIMULATOR=${PX4_SIMULATOR:=gz}
PX4_GZ_WORLD=${PX4_GZ_WORLD:=default}
PX4_SIM_MODEL=${PX4_SIM_MODEL:=x650}

param set-default SIM_GZ_EN 1
param set-default CA_AIRFRAME 0
param set-default CA_ROTOR_COUNT 4

# X650 geometry (tune to CAD)
param set-default CA_ROTOR0_PX  0.165
param set-default CA_ROTOR0_PY  0.275
param set-default CA_ROTOR0_KM  0.05    # ccw
param set-default CA_ROTOR1_PX -0.165
param set-default CA_ROTOR1_PY -0.275
param set-default CA_ROTOR1_KM  0.05    # ccw
param set-default CA_ROTOR2_PX  0.165
param set-default CA_ROTOR2_PY -0.275
param set-default CA_ROTOR2_KM -0.05    # cw
param set-default CA_ROTOR3_PX -0.165
param set-default CA_ROTOR3_PY  0.275
param set-default CA_ROTOR3_KM -0.05    # cw

# Gazebo ESC mapping (adjust if motor order differs)
param set-default SIM_GZ_EC_FUNC1 101
param set-default SIM_GZ_EC_FUNC2 102
param set-default SIM_GZ_EC_FUNC3 103
param set-default SIM_GZ_EC_FUNC4 104

param set-default SIM_GZ_EC_MIN1 150
param set-default SIM_GZ_EC_MIN2 150
param set-default SIM_GZ_EC_MIN3 150
param set-default SIM_GZ_EC_MIN4 150
param set-default SIM_GZ_EC_MAX1 1000
param set-default SIM_GZ_EC_MAX2 1000
param set-default SIM_GZ_EC_MAX3 1000
param set-default SIM_GZ_EC_MAX4 1000

param set-default MPC_THR_HOVER 0.65
param set-default NAV_DLL_ACT 2
```

```
# Permissions + rebuild to package ROMFS
chmod 755 4229_gz_x650
cd PX4-Autopilot
rm -rf build/px4_sitl_default
make px4_sitl gz_x500

# Verify packaging; if missing, copy as fallback
```

```
ls build/px4_sitl_default/rootfs/etc/init.d-posix/airframes | grep 4229 || cp
ROMFS/px4fmu_common/init.d-posix/airframes/4229_gz_x650
build/px4_sitl_default/rootfs/etc/init.d-posix/airframes/
```

## 7. Launch the Simulation

```
# Ensure model path is visible to Gazebo
export GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:$(pwd)/Tools/simulation/gz/models

# Launch PX4 with your custom airframe + model
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 build/px4_sitl_default/bin/px4
# or
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 make px4_sitl gz_x500
```

## 8. Tests & Sanity Checks (Gate Before Next Step)

### 8.1 Description validation

```
check_urdf uav.urdf
gz sdf -k PX4-Autopilot/Tools/simulation/gz/models/x650/model.sdf
```

### 8.2 Resources & topics

```
# Meshes present and lowercase
ls PX4-Autopilot/Tools/simulation/gz/models/x650/meshes/*.stl

# GZ topics
gz topic -l | head -n 30

# Motor commands after arm/takeoff
gz topic -e -t /world/default/model/x650_0/command/motor_speed

# IMU example (path may differ slightly)
gz topic -l | grep imu
```

### 8.3 PX4 console

```
# in pxh>
commander arm
commander takeoff
param show MPC_THR_HOVER
```

### 8.4 QGroundControl motor order test (optional)

Use Vehicle Setup → Motors to verify each motor location. Adjust SDF motorNumber or SIM_GZ_EC_FUNC* mapping if needed.

## 9. Common Problems and Fixes (Observed)

| Symptom | Cause | Fix |
|---|---|---|
| no autostart file found .../airframes/4229_* | Airframe not packaged into ROMFS | Ensure file under ROMFS/.../airframes; chmod +x; rm -rf |
| Failed to load libMavlinkInterface.so | Classic plugin referenced | Remove Classic SDF plugin; PX4 gz_bridge is built-in. |
| Unable to find model://.../*.stl | Wrong URI or uppercase extension | Use model:// URIs; rename .STL→.stl; export GZ_SIM_R |
| Ogre material exception | Duplicate sensors system | Remove model-level gz-sim-sensors-system plugin. |

## 10. Final Clean Run Sequence

```
# 1) URDF from Xacro
ros2 run xacro xacro uav.urdf.xacro > uav.urdf
check_urdf uav.urdf

# 2) SDF from URDF
gz sdf -p uav.urdf > model.sdf
sed -i '/<gz_frame_id>.*<\/gz_frame_id>/d' model.sdf
gz sdf -k model.sdf

# 3) Model placement
mkdir -p PX4-Autopilot/Tools/simulation/gz/models/x650/meshes
cp model.sdf PX4-Autopilot/Tools/simulation/gz/models/x650/
cp meshes/*.stl PX4-Autopilot/Tools/simulation/gz/models/x650/meshes/
export
GZ_SIM_RESOURCE_PATH=$GZ_SIM_RESOURCE_PATH:$(pwd)/PX4-Autopilot/Tools/simulation/gz/models

# 4) Airframe from 4001_gz_x500 + shebang
cd PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes
cp 4001_gz_x500 4229_gz_x650
# ensure first line:
#!/bin/sh
chmod 755 4229_gz_x650

# 5) Rebuild and verify; fallback copy if missing
cd PX4-Autopilot
rm -rf build/px4_sitl_default
make px4_sitl gz_x500
ls build/px4_sitl_default/rootfs/etc/init.d-posix/airframes | grep 4229 || cp
ROMFS/px4fmu_common/init.d-posix/airframes/4229_gz_x650
build/px4_sitl_default/rootfs/etc/init.d-posix/airframes/

# 6) Run
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 build/px4_sitl_default/bin/px4
```

## Appendix A — Fix for 'no autostart file found (4229_*)'

PX4 reports this when the new airframe wasn’t packaged into the runtime rootfs. Solutions:

```
# 1) Confirm the file exists in source
ls ROMFS/px4fmu_common/init.d-posix/airframes | grep 4229

# 2) Clean + rebuild so ROMFS is repacked
make clean
make px4_sitl_default

# 3) If still missing, hot-fix by copying into built rootfs
cp ROMFS/px4fmu_common/init.d-posix/airframes/4229_gz_x650
build/px4_sitl_default/rootfs/etc/init.d-posix/airframes/
```

## Appendix B — Correct Way to Run Your X650

The last argument in the command (e.g., ‘gz_x500’) is a specific simulation target. Using ‘gz_x500’ spawns the X500 even if PX4_GZ_MODEL=x650 is set. Use one of the options below to launch the X650 correctly.

## Option 1 — Reuse the x500 target (simplest)

Override the model at runtime using environment variables (no CMake changes):

```
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 PX4_GZ_WORLD=default
build/px4_sitl_default/bin/px4

# or specify the world explicitly:
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 build/px4_sitl_default/bin/px4 -w
Tools/simulation/gz/worlds/default.sdf
```

## Option 2 — Create a new CMake SITL target (recommended for permanence)

Add a dedicated target called ‘gz_x650’ inside PX4’s CMake configuration so that ‘make px4_sitl gz_x650’ works directly:

```
# Edit PX4-Autopilot/platforms/posix/cmake/sitl_target.cmake

# Locate the section:
set(models
    gz_x500
    gz_iris
    gz_standard_vtol
    ...
)

# Append your model:
set(models
    gz_x500
    gz_x650
    gz_iris
    gz_standard_vtol
    ...
)

# Rebuild so PX4 knows about the new target
rm -rf build/px4_sitl_default
make px4_sitl gz_x650
```

PX4 will then map PX4_GZ_MODEL=x650 to Tools/simulation/gz/models/x650/ and use airframe 4229_gz_x650.

## Option 3 — Manual Gazebo spawn for debugging

Test the model visuals first, then connect PX4:

```
# Terminal 1: Run Gazebo manually
gz sim -v 4 Tools/simulation/gz/worlds/default.sdf
```

```
gz model --spawn-file Tools/simulation/gz/models/x650/model.sdf --model-name x650

# Terminal 2: Run PX4 and connect
PX4_SYS_AUTOSTART=4229 PX4_GZ_MODEL=x650 build/px4_sitl_default/bin/px4
```

## Recommendation

Use Option 2 for a permanent ‘make px4_sitl gz_x650’ target; Option 1 for quick testing; Option 3 for model verification.